

Notice that we have added and subtracted $\beta(1, x)y_2$ in the last line. If you examine the terms containing a factor of y_8 in (5.4.22), you will find that they sum to zero as a consequence of the recurrence relation (5.4.20); similarly for all the other y_k 's down through y_2 . The only surviving terms in (5.4.22) are

$$f(x) = \beta(1, x)F_0(x)y_2 + F_1(x)y_1 + F_0(x)c_0 \quad (5.4.23)$$

Equations (5.4.21) and (5.4.23) are *Clenshaw's recurrence formula* for doing the sum (5.4.19): You make one pass down through the y_k 's using (5.4.21); when you have reached y_2 and y_1 , you apply (5.4.23) to get the desired answer.

Clenshaw's recurrence as written above incorporates the coefficients c_k in a downward order, with k decreasing. At each stage, the effect of all previous c_k 's is "remembered" as two coefficients that multiply the functions F_{k+1} and F_k (ultimately F_0 and F_1). If the functions F_k are small when k is large, *and* if the coefficients c_k are small when k is *small*, then the sum can be dominated by small F_k 's. In this case, the remembered coefficients will involve a delicate cancellation and there can be a catastrophic loss of significance. An example would be to sum the trivial series

$$J_{15}(1) = 0 \times J_0(1) + 0 \times J_1(1) + \dots + 0 \times J_{14}(1) + 1 \times J_{15}(1) \quad (5.4.24)$$

Here J_{15} , which is tiny, ends up represented as a canceling linear combination of J_0 and J_1 , which are of order unity.

The solution in such cases is to use an alternative Clenshaw recurrence that incorporates the c_k 's in an upward direction. The relevant equations are

$$y_{-2} = y_{-1} = 0 \quad (5.4.25)$$

$$y_k = \frac{1}{\beta(k+1, x)}[y_{k-2} - \alpha(k, x)y_{k-1} - c_k], \quad k = 0, 1, \dots, N-1 \quad (5.4.26)$$

$$f(x) = c_N F_N(x) - \beta(N, x)F_{N-1}(x)y_{N-1} - F_N(x)y_{N-2} \quad (5.4.27)$$

The rare case where equations (5.4.25) – (5.4.27) should be used instead of equations (5.4.21) and (5.4.23) can be detected automatically by testing whether the operands in the first sum in (5.4.23) are opposite in sign and nearly equal in magnitude. Other than in this special case, Clenshaw's recurrence is always stable, independent of whether the recurrence for the functions F_k is stable in the upward or downward direction.

5.4.3 Parallel Evaluation of Linear Recurrence Relations

When desirable, linear recurrence relations can be evaluated with a lot of parallelism. Consider the general first-order linear recurrence relation

$$u_j = a_j + b_{j-1}u_{j-1}, \quad j = 2, 3, \dots, n \quad (5.4.28)$$

with initial value $u_1 = a_1$. To parallelize the recurrence, we can employ the powerful general strategy of *recursive doubling*. Write down equation (5.4.28) for $2j$ and for $2j-1$:

$$\begin{aligned} u_{2j} &= a_{2j} + b_{2j-1}u_{2j-1} \\ u_{2j-1} &= a_{2j-1} + b_{2j-2}u_{2j-2} \end{aligned} \quad (5.4.29)$$

Substitute the second of these equations into the first to eliminate u_{2j-1} and get

$$u_{2j} = (a_{2j} + a_{2j-1}b_{2j-1}) + (b_{2j-2}b_{2j-1})u_{2j-2} \quad (5.4.30)$$

This is a new recurrence of the same form as (5.4.28) but over only the even u_j , and hence involving only $n/2$ terms. Clearly we can continue this process recursively, halving the number of terms in the recurrence at each stage, until we are left with a recurrence of length 1 or 2 that we can do explicitly. Each time we finish a subpart of the recursion, we fill in the odd terms in the recurrence, using the second equation in (5.4.29). In practice, it's even easier than it sounds. The total number of operations is the same as for serial evaluation, but they are done in about $\log_2 n$ parallel steps.

There is a variant of recursive doubling, called *cyclic reduction*, that can be implemented with a straightforward iteration loop instead of a recursive procedure [6]. Here we start by writing down the recurrence (5.4.28) for all adjacent terms u_j and u_{j-1} (not just the even ones, as before). Eliminating u_{j-1} , just as in equation (5.4.30), gives

$$u_j = (a_j + a_{j-1}b_{j-1}) + (b_{j-2}b_{j-1})u_{j-2} \quad (5.4.31)$$

which is a first-order recurrence with new coefficients a'_j and b'_j . Repeating this process gives successive formulas for u_j in terms of $u_{j-2}, u_{j-4}, u_{j-8}, \dots$. The procedure terminates when we reach u_{j-n} (for n a power of 2), which is zero for all j . Thus the last step gives u_j equal to the last set of a'_j 's.

In cyclic reduction, the length of the vector u_j that is updated at each stage does not decrease by a factor of 2 at each stage, but rather only decreases from $\sim n$ to $\sim n/2$ during all $\log_2 n$ stages. Thus the total number of operations carried out is $O(n \log n)$, as opposed to $O(n)$ for recursive doubling. Whether this is important depends on the details of the computer's architecture.

Second-order recurrence relations can also be parallelized. Consider the second-order recurrence relation

$$y_j = a_j + b_{j-2}y_{j-1} + c_{j-2}y_{j-2}, \quad j = 3, 4, \dots, n \quad (5.4.32)$$

with initial values

$$y_1 = a_1, \quad y_2 = a_2 \quad (5.4.33)$$

With this numbering scheme, you supply coefficients $a_1, \dots, a_n$, $b_1, \dots, b_{n-2}$, and $c_1, \dots, c_{n-2}$. Rewrite the recurrence relation in the form [6]

$$\begin{pmatrix} y_j \\ y_{j+1} \end{pmatrix} = \begin{pmatrix} 0 \\ a_{j+1} \end{pmatrix} + \begin{pmatrix} 0 & 1 \\ c_{j-1} & b_{j-1} \end{pmatrix} \begin{pmatrix} y_{j-1} \\ y_j \end{pmatrix}, \quad j = 2, \dots, n-1 \quad (5.4.34)$$

that is,

$$\mathbf{u}_j = \mathbf{a}_j + \mathbf{b}_{j-1} \cdot \mathbf{u}_{j-1}, \quad j = 2, \dots, n-1 \quad (5.4.35)$$

where

$$\mathbf{u}_j = \begin{pmatrix} y_j \\ y_{j+1} \end{pmatrix}, \quad \mathbf{a}_j = \begin{pmatrix} 0 \\ a_{j+1} \end{pmatrix}, \quad \mathbf{b}_{j-1} = \begin{pmatrix} 0 & 1 \\ c_{j-1} & b_{j-1} \end{pmatrix} \quad (5.4.36)$$

and

$$\mathbf{u}_1 = \mathbf{a}_1 = \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} a_1 \\ a_2 \end{pmatrix} \quad (5.4.37)$$

This is a first-order recurrence relation for the vectors $\mathbf{u}_j$ and can be solved by either of the algorithms described above. The only difference is that the multiplications are matrix multiplications with the 2×2 matrices $\mathbf{b}_j$. After the first recursive call, the zeros in $\mathbf{a}$ and $\mathbf{b}$ are lost, so we have to write the routine for general two-dimensional vectors and matrices. Note that this algorithm does not avoid the potential instability problems associated with second-order recurrences that were discussed in §5.4.1. Also note that the algorithm generalizes in the obvious way to higher-order recurrences: An n th-order recurrence can be written as a first-order recurrence involving vectors and matrices of dimension n .

CITED REFERENCES AND FURTHER READING:

- Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at <http://www.nist.gov/aands>, pp. xiii, 697.[1]
- Gautschi, W. 1967, "Computational Aspects of Three-Term Recurrence Relations," *SIAM Review*, vol. 9, pp. 24–82.[2]
- Lakshmikantham, V., and Trigiante, D. 1988, *Theory of Difference Equations: Numerical Methods and Applications* (San Diego: Academic Press).[3]
- Acton, F.S. 1970, *Numerical Methods That Work*, 1990, corrected edition (Washington, DC: Mathematical Association of America), pp. 20ff.[4]
- Clenshaw, C.W. 1962, *Mathematical Tables*, vol. 5, National Physical Laboratory (London: H.M. Stationery Office).[5]
- Dahlquist, G., and Björck, A. 1974, *Numerical Methods* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2003 (New York: Dover), §4.4.3, p. 111.
- Goodwin, E.T. (ed.) 1961, *Modern Computing Methods*, 2nd ed. (New York: Philosophical Library), p. 76.
- Hockney, R.W., and Jesshope, C.R. 1988, *Parallel Computers 2: Architecture, Programming, and Algorithms* (Bristol and Philadelphia: Adam Hilger), §5.2.4 and §5.4.2.[6]

5.5 Complex Arithmetic

Since C++ has a built-in class `complex`, you can generally let the compiler and the class library take care of complex arithmetic for you. Generally, but not always. For a program with only a small number of complex operations, you may want to code these yourself, in-line. Or, you may find that your compiler is not up to snuff: It is disconcertingly common to encounter complex operations that produce overflows or underflows when both the complex operands and the complex result are perfectly representable. This occurs, we think, because software companies mistake the implementation of complex arithmetic for a completely trivial task, not requiring any particular finesse.

Actually, complex arithmetic is not *quite* trivial. Addition and subtraction are done in the obvious way, performing the operation separately on the real and imaginary parts of the operands. Multiplication can also be done in the obvious way, with four multiplications, one addition, and one subtraction:

$$(a + ib)(c + id) = (ac - bd) + i(bc + ad) \quad (5.5.1)$$

(the addition sign before the i doesn't count; it just separates the real and imaginary parts notationally). But it is sometimes faster to multiply via

$$(a + ib)(c + id) = (ac - bd) + i[(a + b)(c + d) - ac - bd] \quad (5.5.2)$$

which has only three multiplications (ac , bd , $(a + b)(c + d)$), plus two additions and three subtractions. The total operations count is higher by two, but multiplication is a slow operation on some machines.

While it is true that intermediate results in equations (5.5.1) and (5.5.2) can overflow even when the final result is representable, this happens only when the final

answer is on the edge of representability. Not so for the complex modulus, if you or your compiler is misguided enough to compute it as

$$|a + ib| = \sqrt{a^2 + b^2} \quad (\text{bad!}) \quad (5.5.3)$$

whose intermediate result will overflow if either a or b is as large as the square root of the largest representable number (e.g., 10^{19} as compared to 10^{38}). The right way to do the calculation is

$$|a + ib| = \begin{cases} |a|\sqrt{1 + (b/a)^2} & |a| \geq |b| \\ |b|\sqrt{1 + (a/b)^2} & |a| < |b| \end{cases} \quad (5.5.4)$$

Complex division should use a similar trick to prevent avoidable overflow, underflow, or loss of precision:

$$\frac{a + ib}{c + id} = \begin{cases} \frac{[a + b(d/c)] + i[b - a(d/c)]}{c + d(d/c)} & |c| \geq |d| \\ \frac{[a(c/d) + b] + i[b(c/d) - a]}{c(c/d) + d} & |c| < |d| \end{cases} \quad (5.5.5)$$

Of course you should calculate repeated subexpressions, like c/d or d/c , only once.

Complex square root is even more complicated, since we must both guard intermediate results and also enforce a chosen branch cut (here taken to be the negative real axis). To take the square root of $c + id$, first compute

$$w \equiv \begin{cases} 0 & c = d = 0 \\ \sqrt{|c|} \sqrt{\frac{1 + \sqrt{1 + (d/c)^2}}{2}} & |c| \geq |d| \\ \sqrt{|d|} \sqrt{\frac{|c/d| + \sqrt{1 + (c/d)^2}}{2}} & |c| < |d| \end{cases} \quad (5.5.6)$$

Then the answer is

$$\sqrt{c + id} = \begin{cases} 0 & w = 0 \\ w + i \left(\frac{d}{2w} \right) & w \neq 0, c \geq 0 \\ \frac{|d|}{2w} + iw & w \neq 0, c < 0, d \geq 0 \\ \frac{|d|}{2w} - iw & w \neq 0, c < 0, d < 0 \end{cases} \quad (5.5.7)$$

CITED REFERENCES AND FURTHER READING:

- Midy, P., and Yakovlev, Y. 1991, "Computing Some Elementary Functions of a Complex Variable," *Mathematics and Computers in Simulation*, vol. 33, pp. 33–49.
- Knuth, D.E. 1997, *Seminumerical Algorithms*, 3rd ed., vol. 2 of *The Art of Computer Programming* (Reading, MA: Addison-Wesley) [see solutions to exercises 4.2.1.16 and 4.6.4.41].